

Fundamentals of Programming

CS-114

Week - 11

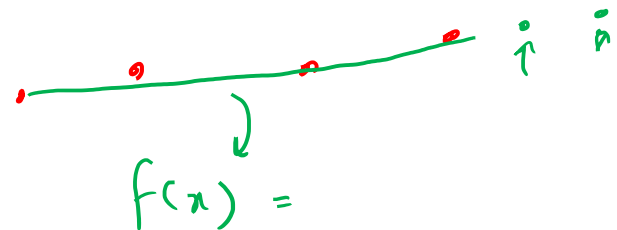
Multi-dimensional Arrays

Multi Dimensional Arrays (Matrices)

$a[0] \dots a[3]$ $a[4] \rightarrow \text{garbage}$
 $a[0] \dots a[3]$

- An array can have two or more dimensions. This permits them to model multidimensional objects, such as graph paper or the computer display screen.
- They are often used to represent **tables of values** consisting of information arranged in **rows** and **columns**.

$\Rightarrow$ interpolation $\rightarrow$



$$X = \begin{bmatrix} 3 & 6 & 9 \\ 1 & 2 & -9 \\ 3 & 6 & 10 \end{bmatrix}$$

→ 2 Dimensional array → matrix

→ nested loops

i ↓

		j →	3x3
① → 0	↓	0	↓
→	3	6	9
	0,0	0,1	0,2
② → 1	↓	1	↓
→	1	2	-9
	1,0	1,1	1,2
③ → 2	↓	2	↓
→	3	6	10
	2,0	2,1	2,2

for loop
0 → 2

for loop
0 → 2

```
for (int i=0; i<3; i++)
{
```

← i → rows

```
for (int j=0; j<3; j++)
```

← j → cols

```
cout << X[i][j];
```

row col

```
}
```

Multi Dimensional Arrays (Matrices)

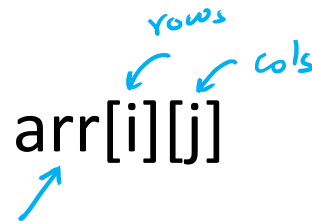
- To identify a particular element, we must specify two subscripts. By convention,

array_name[row_number][col_number]



- For example,

arr[i][j]



Multiple-Subscripted Arrays

- Multiple subscripted arrays
 - Tables with rows and columns (m by n array)
 - Like matrices: specify row, then column

a →

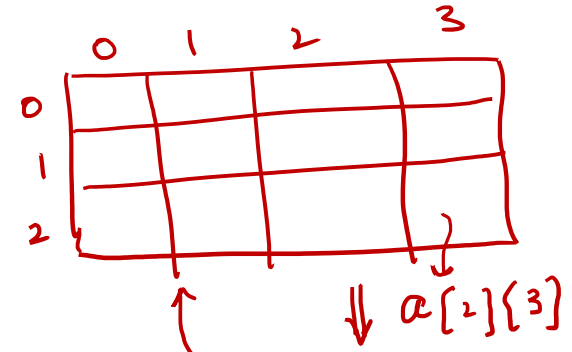
	Column	Column 1	Column	Column 3
Row 0	a[0][0]	a[0][1]	a[0][2]	a[0][3]
Row 1	a[1][0]	a[1][1]	a[1][2]	a[1][3]
Row 2	a[2][0]	a[2][1]	a[2][2]	a[2][3]

↓

Array name Row subscript column subscript

6.9 Multiple-Subscripted Arrays

- `int a[3][4];` ← declaration



	Column 0	Column 1	Column 2	Column 3
Row 0	<code>a[0][0]</code>	<code>a[0][1]</code>	<code>a[0][2]</code>	<code>a[0][3]</code>
Row 1	<code>a[1][0]</code>	<code>a[1][1]</code>	<code>a[1][2]</code>	<code>a[1][3]</code>
Row 2	<code>a[2][0]</code>	<code>a[2][1]</code>	<code>a[2][2]</code>	<code>a[2][3]</code>

`int a[3][4];`

`cout << a[2][3];`

Array name Row subscript Column subscript

6.9 Multiple-Subscripted Arrays

- Initialization

- `int b[2][2] = { { 1, 2 }, { 3, 4 } };`
- Initializers grouped by row in braces

`int a[2][2];`

`int b[2][2] = { { 1, 2 }, { 3, 4 } };`

Diagram illustrating the 2x2 array structure with row and column indices:

$b[0][0]$	$b[0][1]$
1	2
$b[1][0]$	$b[1][1]$
3	4

Labels: 2×2 , 1st row, 2nd row, row, column.

```
for (int i = 0; i < 2; i++)
{
    for (int j = 0; j < 2; j++)
    {
        cin >> a[i][j];
    }
}
```

↑
4 values input

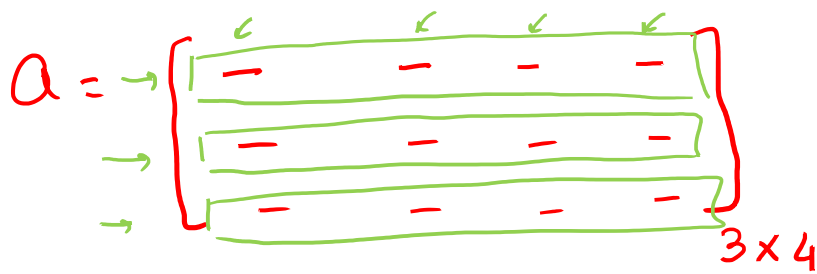
Diagram illustrating the 2x2 array structure with row and column indices:

$b[0][0]$	$b[0][1]$
1	2
$b[1][0]$	$b[1][1]$
3	4

Labels: 2×2 , 1st row, 2nd row, row, column.

Coordinates for each element:

- $i = 0$, $j = 0, 1$: $(0, 0)$, $(0, 1)$
- $i = 1$, $j = 0, 1$: $(1, 0)$, $(1, 1)$



`int a[3][4];` $\swarrow$ $0,0 \rightarrow 2,3$

```

for (int i=0; i<3; i++)
{
    for (int j=0; j<4; j++)
    {
        cout << a[i][j] << endl;
        cin >> a[i][j];
    }
}

```

Index

$a[2][2] = 10;$

$a[1][3] = 6;$

$\rightarrow i=0$

$j=0$	$j=1$	$j=2$	$j=3$
9	10	7	6
6	-9	3	7
11	15	9	3

$2,3$ 3×4

3	6	9	11
9	19	3	9

```

cout << a[2][3];
{
    cout << a[i][j] << " ";
}
cout << endl;

```


6.9 Multiple-Subscripted Arrays

- Initialization

- `int b[2][2] = { { 1, 2 }, { 3, 4 } };`
- Initializers grouped by row in braces

1	2
3	4

- If not enough, unspecified elements set to zero

`int b[2][2] = { { 1 }, { 3, 4 } };`

1	0
3	4

- Referencing elements

- Specify row, then column
`cout<<b[0][1];`

`int b[ ][ ] = { { 3, 3 }, { 3, 3 } }`

`int array[ ][ ] = { ..., ..., ..., ... };`

Initializing a 2D Array

- int matrix [10] [2] = { {59 , 78}, ← row 0
{14 , 17}, ← row 1
⇒ {68 , 28},
{32 , 45},
{ 5 , 14},
{12 , 15},
{ 6, 2},
{ 22, 1},
{14 , 16},
row 9 → { 2, 58} };
 ↑ ↑
 col 0 col 1

- The values used to initialize an array are separated by commas and surrounded by braces.

$$a = \begin{bmatrix} 3 & 9 & 7 \\ 2 & 6 & 5 \\ 9 & 6 & 1 \end{bmatrix} \Rightarrow b = a^T = \begin{bmatrix} 3 & 2 & 9 \\ 9 & 6 & 6 \\ 7 & 5 & 1 \end{bmatrix}$$

$\text{int } a[3][3] = \{ \{ 3, 9, 7 \}, \{ 2, 6, 5 \}, \{ 9, 6, 1 \} \};$
size size
 $\text{int } b[3][3];$

for (int j = 0; j < 3; j++)

{

for (int i = 0; i < 3; i++)

{

$b[j][i] = a[i][j];$
row col row col

}

		j						i		
i	a	3	9	7	j	3	2	9	i	
		2	6	5		9	6	6		
		9	6	1		7	5	1		

$a[\text{row}_i][\text{col}_j]$

$b = a^T [\text{col}_j][\text{row}_i]$

↓

$b = [\text{row}][\text{col}]$

2-D Array (Matrix)

```
int matrix [4] [4] = { {59, 78,14, 17},{68, 28,32, 45},  
                      {5, 14,12, 15} ,{6, 2, 22, 1} };
```

(row = 2, col = 0)



59 (0,0)	78 (0,1)	14 (0,2)	17 (0,3)
68 (1,0)	28 (1,1)	32 (1,2)	45 (1,3)
5 (2,0)	14 (2,1)	12 (2,2)	15 (2,3)
6 (3,0)	2 (3,1)	22 (3,2)	1 (3,3)

4 x 4 Matrix

Reading a 2D Array

```
int matrix [4][4]={59,78,14,17},{68,28,32,45},  
                  {5, 14,12,15},{6,2, 22, 1}};
```

```
for(int i=0;i<4;i++)  
{  
    for(int j=0;j<4;j++)  
    {  
        cout<<matrix[i][j]<<"\t";  
    }  
    cout<<endl;  
}
```

Example

- Write a matrix with following entries

int a[6][2] = { {12, 12}, {54, 34}, {16, 3}, {12, 6}, {43, 23}, {1, 2} };

a =

12	12
54	34
16	3
12	6
43	23
1	2

6x2

```
int size;
```

```
cin >> size;
```

```
int a[size][size];
```

```
for (int i=0; i<size; i++)
{
```

```
    for (int j=0; j<size; j++)
```

```
    {
```

```
        if (i == j)
```

```
            a[i][j] = 1;
```

```
        else
```

```
            a[i][j] = 0;
```

```
    }
```

```
}
```

for elements with
even rows & cols
put 1 else 0

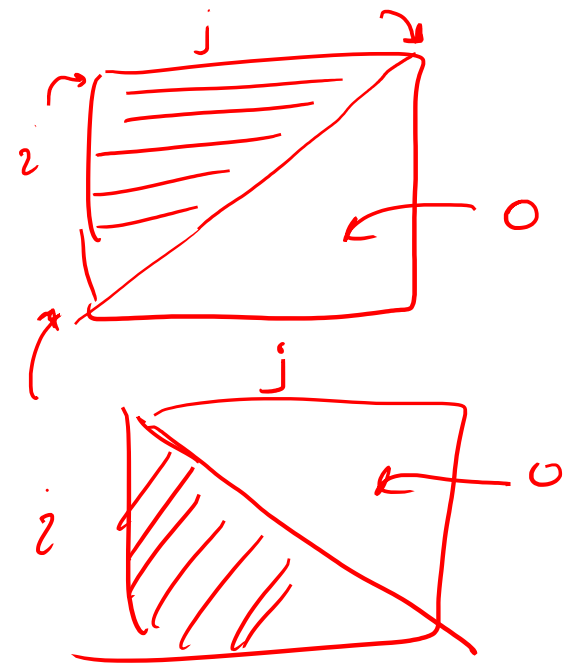
if (i % 2 == 0 && j % 2 == 0)

for identity

```
int size;
```

```
cin >> size;
```

```
for (int i=0;
```



~~char~~

$a[] = \{ 'A', 'l', 'i', '\0' \};$

2D String

characters

3 character

$\rightarrow s'se \rightarrow 4$

char a = 's';

array of characters

↓
string

1001

A	l	i	\0						
---	---	---	----	--	--	--	--	--	--

1011

A	b	b	a	s	\0				
---	---	---	---	---	----	--	--	--	--

1021

A	l	i	n	a	\0				
---	---	---	---	---	----	--	--	--	--

1031

A	y	e	s	h	a	\0			
---	---	---	---	---	---	----	--	--	--

1041

B	a	n	o	\0					
---	---	---	---	----	--	--	--	--	--

char $\rightarrow a[] = "Ali";$

2D array of characters

A	l	i	\0
---	---	---	----

null
Character/operator

```

1  /* Fig. 6.21: fig06_21.c
2      Initializing multidimensional arrays */
3  #include <stdio.h>
4
5  void printArray( const int a[][ 3 ] ); /* function prototype */
6
7  /* function main begins program execution */
8  int main()
9  {
10     /* initialize array1, array2, array3 */
11     int array1[ 2 ][ 3 ] = { { 1, 2, 3 }, { 4, 5, 6 } };
12     int array2[ 2 ][ 3 ] = { 1, 2, 3, 4, 5 };
13     int array3[ 2 ][ 3 ] = { { 1, 2 }, { 4 } };
14
15     cout<<"Values in array1 by row are:\n" ;
16     printArray( array1 );
17
18     cout<<"Values in array2 by row are:\n" ;
19     printArray( array2 );
20
21     cout<<"Values in array3 by row are:\n" ;
22     printArray( array3 );
23
24     return 0; /* indicates successful termination */
25
26 } /* end main */
27

```

Handwritten annotations:

- Red arrow from "no output" points to the `void` keyword in line 5.
- Red arrow from "input array" points to the `int a[][3]` in line 5.
- Red arrows point to the `const` and `3` in line 5, which are crossed out with red slashes.

```

28 /* function to output array with two rows and three columns */
29 void printArray( const int a[ ][ ] ) ← function body
30 {
31     int i; /* counter */
32     int j; /* counter */
33
34     /* loop through rows */
35     for ( i = 0; i <= 1; i++ ) {
36
37         /* output column values */
38         for ( j = 0; j <= 2; j++ ) {
39             cout<<a[ i ][ j ] ;
40         } /* end inner for */
41
42         cout<<"\n" ; /* start new line of output */
43     } /* end outer for */
44
45 } /* end function printArray */

```

Program Output

```

Values in array1 by row are:
1 2 3
4 5 6
Values in array2 by row are:
1 2 3
4 5 0
Values in array3 by row are:
1 2 0
4 0 0

```

Examples Using Arrays

- Character arrays

It is used in programming for storing and manipulating text, such as words, names and sentences.

int a[] = { 'a', 'h', 'm', 'e', 'd', '\0' };

– Character arrays can be initialized using string literals

⇒ char string1[] = "first";

- Null character *'\0'* terminates strings

- string1 actually has 6 elements

– It is equivalent to

char string1[] = { 'f', 'i', 'r', 's', 't', '\0' };

– Can access individual characters

string1[3] is character 's'

char a[5];

cout << string1[2]; // r

Character arrays

- Like all other variables and arrays

```
char name[5];
```

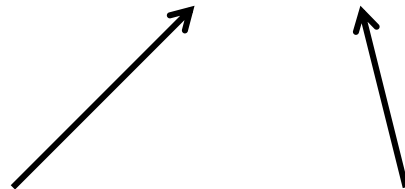
```
cin >> name;
```

```
cout<< "you entered " << name;
```

- The << operator displays the output until it encounters a null character.
- In case of a string variable, the cin operator places the null automatically.

Multi-Dimensional Strings (Array of Strings)

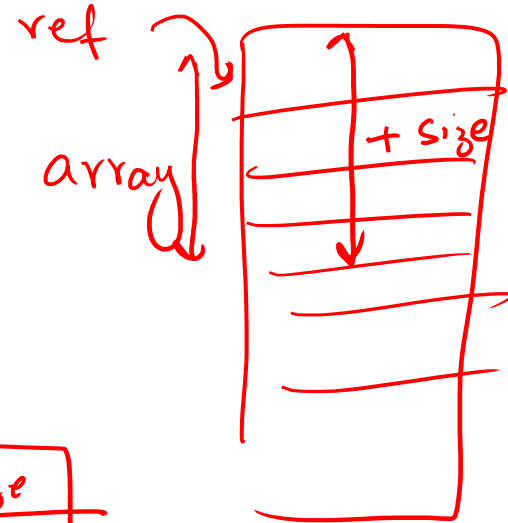
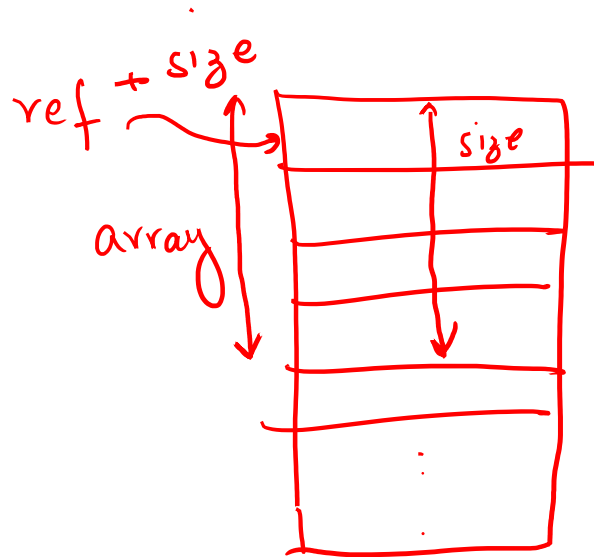
- char stu_lists [5] [10] = {"Ali",
"Abbas",
"Alina",
"Ayesha",
"Bano",
};



A	L	I		
A	b	b	a	s
A	L	I	n	a

Passing Elements of an Array to a Function

- Arrays are passed
 - By Reference ←
 - By value



Passing Arrays to Functions

- Passing arrays

- To pass an array argument to a function, specify the name of the array without any brackets

```
int myArray[ 24 ];
```

```
myFunction( myArray, 24 );
```

- Array size usually passed to function

- Arrays passed **call-by-reference**

- Name of array is address of first element

- Function knows where the array is stored

- Modifies original memory locations

- Passing array elements

- Passed by call-by-value

- Pass subscripted name (i.e., myArray[3]) to function

A diagram illustrating the memory layout of an array. It shows a vertical sequence of elements starting from $a[0]$ at the top and ending at $a[size-1]$ at the bottom. A vertical line on the right side of the sequence is labeled "size", indicating the total number of elements in the array.

Passing Arrays to Functions

- Function prototype

```
void modifyArray( int b[], int arraySize );
```

- Parameter names optional in prototype only

- `int b[]` could be written `int []`
- `int arraySize` could be simply `int`

- The following are equivalent prototypes:

prototyping → `void modifyArray( int b[], int arraySize );`
→ `void modifyArray( int [], int );`

integer array ← *integers* ← *integer (size)*